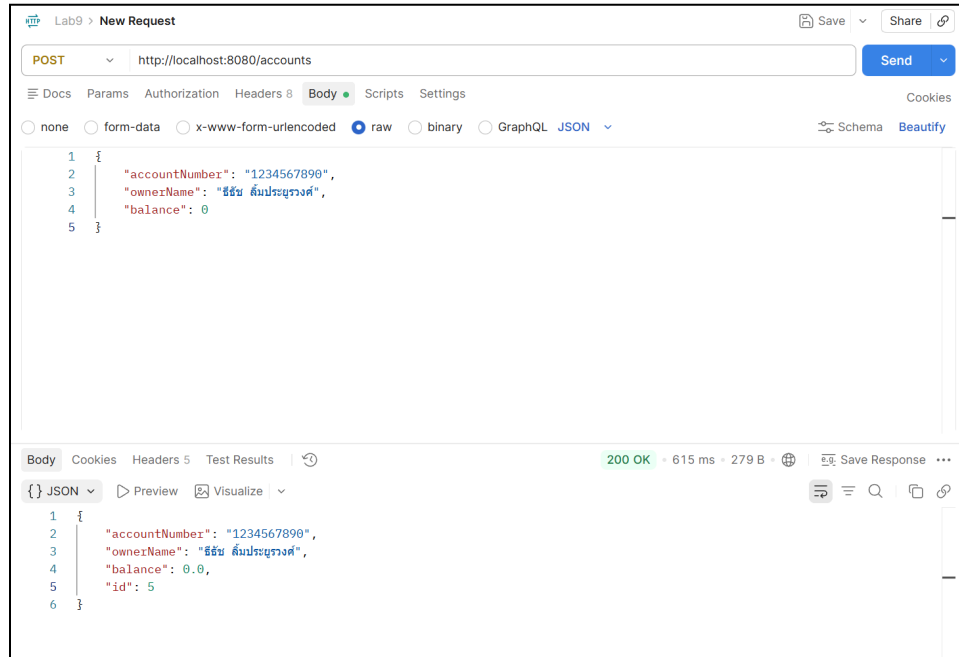


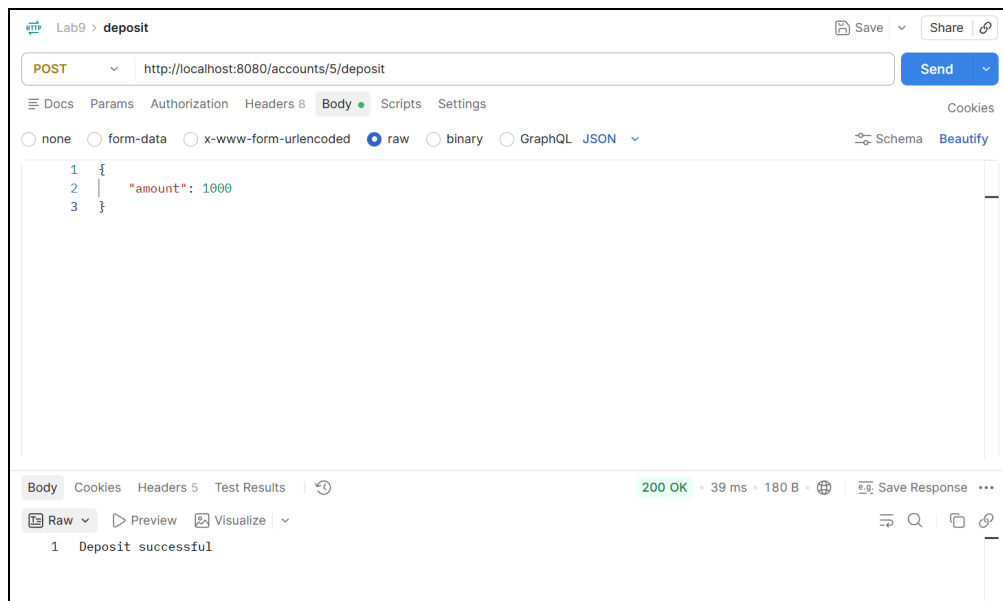
Lab9 Transactional Report

1. การสร้าง Account



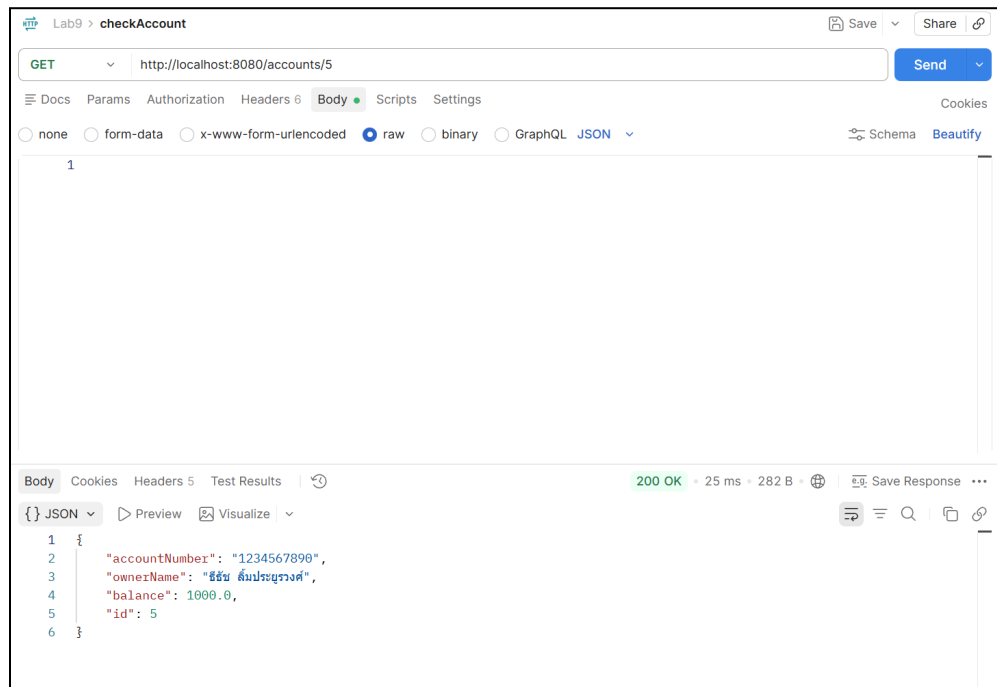
- สร้าง Account สำเร็จด้วยการส่งข้อมูลรูปแบบของ Response และ JSON กลับมาในรูปแบบของ JSON

2. การฝากเงินสำเร็จ



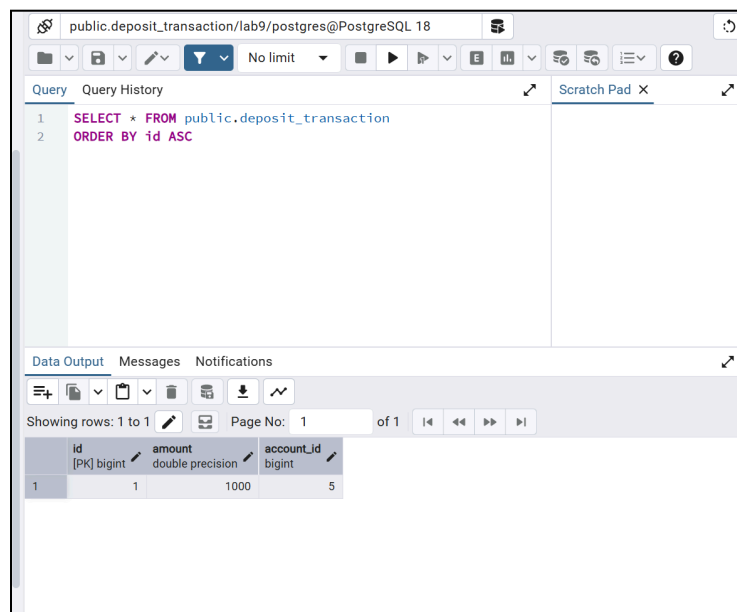
- ส่งจำนวนเงินไปให้ BackEnd ในรูปแบบของ JSON โดยฝากไปยัง account 5 ตาม PathVariable และเมื่อฝากเงินสำเร็จ BackEnd จะ Response กลับมาว่า Deposit Successful

3. ตรวจสอบ Account



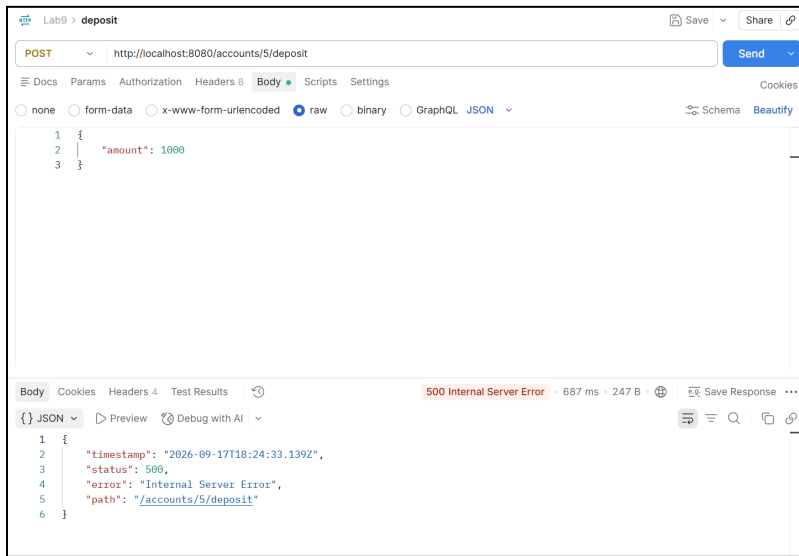
- ระบบจะส่ง Response ข้อมูลของ AccountId ที่เลือกกลับมา

4. ตรวจสอบ DepositTransaction

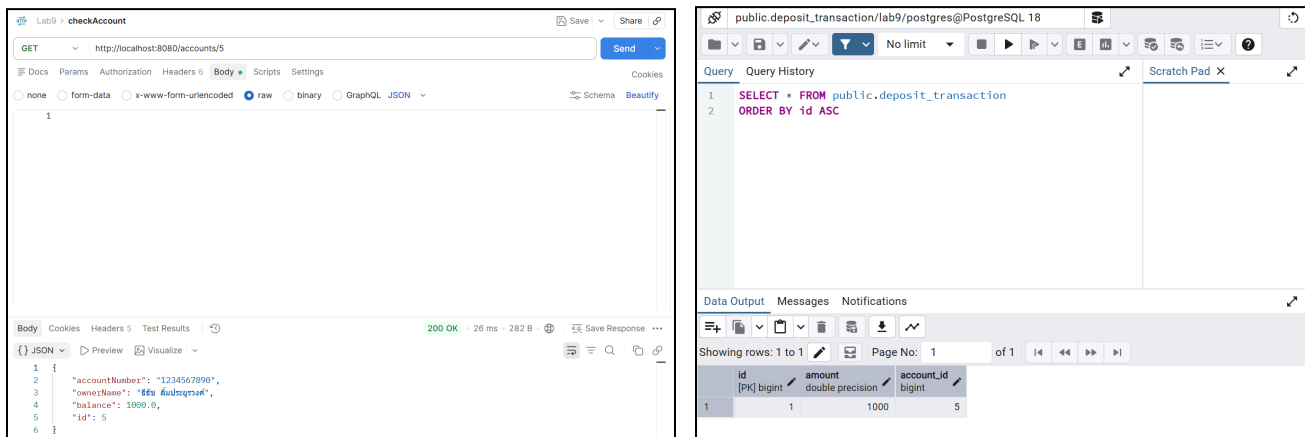


- ระบบจะส่ง Response ข้อมูลของ AccountId ที่เลือกกลับมา

5. ทดลอง Rollback



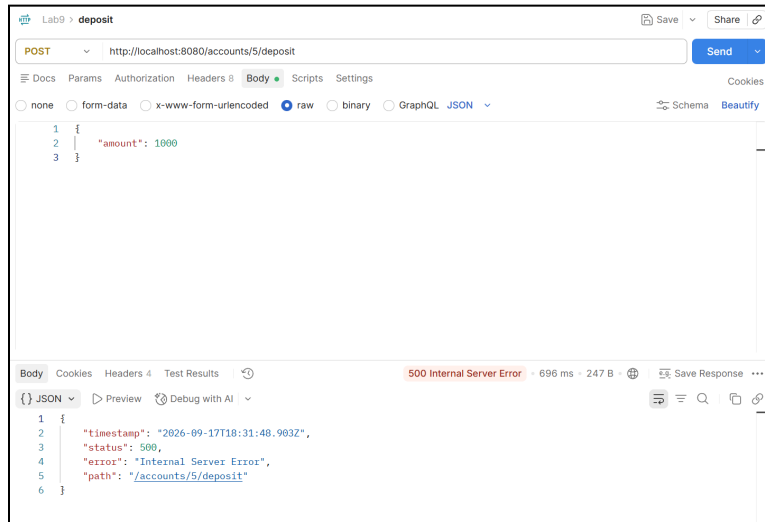
(ระบบเกิดการ error ขึ้น)



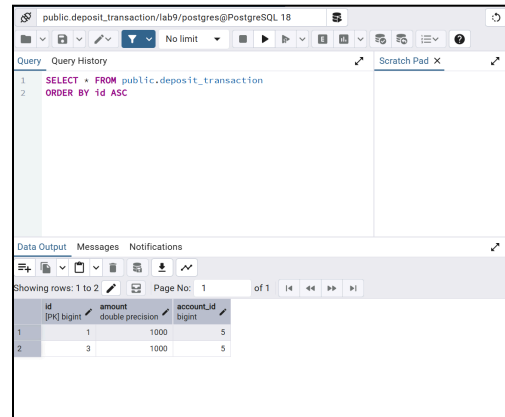
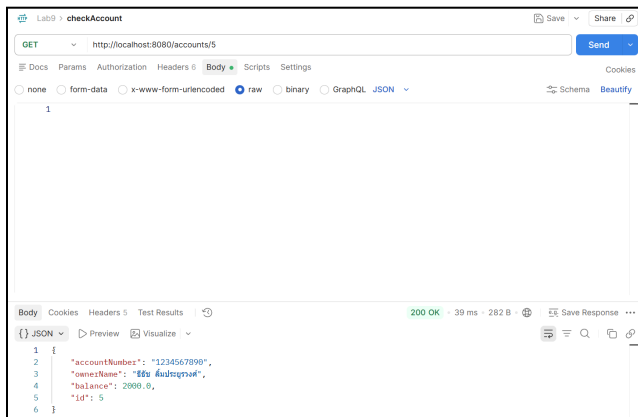
(ระบบ Rollback ขอบกพร่อง ทำให้ข้อมูลคงค่าเดิมไว้)

- จากการใส่ error ไว้ในโค้ด ทำให้ Request นี้ไม่สำเร็จ การแก้ไขจึงถูกยกเลิกและคงค่าเดิมไว้

6. เปรียบเทียบระหว่างมี @Transactional กับไม่มี



(ระบบเกิดการ error ขึ้น)



(ระบบไม่ทำการ Rollback หลังลบ @Transactional ออก ทำให้ค่าใหม่ถูกบันทึกเข้าฐานข้อมูล)

- พอไม่มี @Transactional ถึงแม้จะ throwError ก็ตาม ระบบก็จะยังทำงานไปที่ละบรรทัดโดยไม่เกิดการ Rollback ขึ้น

7. คำถามท้ายบท

1. @Transactional มีหน้าที่อะไร?

ตอบ: ควบคุมการทำงานของ Database ให้เป็น Transaction เดียวกัน และทำการ Rollback เมื่อระบบเกิดข้อบกพร่อง

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ตอบ: เพื่อให้เก็บข้อมูลได้อย่างครบถ้วน และแก้ไขได้หากเกิดข้อผิดพลาด

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

ตอบ: Commit คือการยืนยันการเปลี่ยนแปลง ส่วนการ Rollback คือการย้อนค่ากลับหรือยกเลิกการเปลี่ยนแปลง

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ตอบ: OneToMany, 1 : *

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ตอบ: ระบบจะ Rollback การเปลี่ยนแปลงที่กระทำล่าสุดและคงค่าเดิมเอาไว้

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ตอบ: เมื่อเกิด Error ข้อมูลบางส่วนอาจบันทึกไว้แล้ว ทำให้ข้อมูลไม่สมบูรณ์